

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 14
“SORTING”



DISUSUN OLEH:

RAFI RAMADHAN

109082500140

S1 IF-13-02

DOSEN:

Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

Pengurutan data (*sorting*) merupakan proses menyusun sekumpulan data acak menjadi terstruktur dan teratur, baik itu secara membesar (*ascending*) maupun mengecil (*descending*). Dalam pemrograman, pengurutan sangat penting untuk mempermudah pencarian data dan analisis informasi, baik pada *array* bertipe data dasar maupun tipe data bentukan seperti *struct*. Terdapat berbagai macam algoritma pengurutan yang memanfaatkan logika pencarian data, dan dua di antaranya yang paling fundamental adalah Pengurutan Seleksi (*Selection Sort*) dan Pengurutan Sisipan (*Insertion Sort*).

Algoritma *Selection Sort* bekerja dengan ide dasar mencari nilai ekstrim (seperti nilai terkecil atau terbesar) pada sekumpulan data yang belum teratur, kemudian meletakkannya secara langsung pada posisi yang seharusnya. Secara garis besar, algoritma ini melibatkan dua proses utama: pencarian indeks nilai ekstrim dan proses pertukaran dua nilai (*swap*). Sebagai contoh, pada pengurutan *ascending*, algoritma akan mencari nilai terkecil di rentang data yang tersisa, menukarnya dengan elemen paling kiri pada rentang tersebut, dan mengulangi proses ini terus-menerus hingga seluruh data habis dan tersusun rapi.

Di sisi lain, algoritma *Insertion Sort* memiliki pendekatan yang berbeda karena tidak berfokus pada pencarian nilai ekstrim secara keseluruhan terlebih dahulu. Ide utamanya adalah mengambil suatu nilai secara sekuensial, kemudian menyisipkannya pada posisi yang tepat di antara elemen-elemen yang sebelumnya sudah teratur. Proses penyisipan ini dilakukan dengan cara menggeser data yang sudah teratur ke satu sisi untuk menciptakan satu ruang kosong, sehingga data yang baru dapat dimasukkan tanpa merusak keterurutan susunan sebelumnya. Langkah penggeseran dan penyisipan ini diulangi untuk setiap data individu yang belum teratur.

Meskipun memiliki pendekatan logika yang berbeda, baik *Selection Sort* maupun *Insertion Sort* sama-sama sangat cocok diterapkan pada kasus pengelolaan sekumpulan data terstruktur. Pemahaman mendalam terkait cara kerja kedua algoritma dasar ini dalam membandingkan, menukar, dan menggeser posisi elemen data akan memberikan pondasi logika algoritmik yang kuat dalam membangun program yang efisien.

SOAL LATIHAN

1. Rumah Kerabat

Source Code:

```
package main

import "fmt"

type arrInt [1000000]int

func selectionSort(T *arrInt, n int) {
    var i, j, idx_min, t int
    i = 1
    for i <= n-1 {
        idx_min = i - 1
        j = i
        for j < n {
            if T[idx_min] > T[j] {
                idx_min = j
            }
            j = j + 1
        }
        t = T[idx_min]
        T[idx_min] = T[i-1]
        T[i-1] = t
        i = i + 1
    }
}

func main() {
    var n, m int
    var data arrInt

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&m)

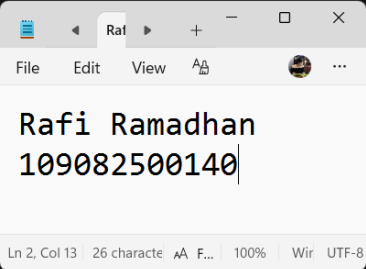
        for j := 0; j < m; j++ {
            fmt.Scan(&data[j])
        }

        selectionSort(&data, m)

        for j := 0; j < m; j++ {
            if j < m-1 {
                fmt.Printf("%d ", data[j])
            } else {
                fmt.Printf("%d\n", data[j])
            }
        }
    }
}
```

Output:

```
7 func selectionSort(T *arrInt, n int) {
16     }
17     j = j + 1
18 }
19 t = T[idx_min]
20 T[idx_min] = T[i-1]
21 T[i-1] = t
22 i = i + 1
23 }
24 }
25
26 func main() {
27     var n, m int
28     var data arrInt
29
30     fmt.Scan(&n)
31
32     for i := 0; i < n; i++ {
33         fmt.Scan(&m)
34
35         for j := 0; j < m; j++ {
36             fmt.Scan(&data[j])
37         }
38
39         selectionSort(&data, m)
40
41         for j := 0; j < m; j++ {
42             if j < m-1 {
43                 fmt.Printf("%d ", data[j])
44             } else {
45                 fmt.Printf("%d\n", data[j])
46             }
47         }
48     }
49 }
```



Deskripsi Program:

Program rumahkerabat ini berfungsi untuk mengatur rute kunjungan Hercules dengan membaca jumlah total daerah n yang akan dikunjungi, lalu secara berulang memproses data dari masing-masing daerah tersebut. Untuk setiap daerah, program membaca jumlah kerabat m diikuti dengan sekumpulan nomor rumah mereka yang disimpan ke dalam sebuah array. Array tersebut kemudian diurutkan secara membesar (ascending) menggunakan algoritma Selection Sort, yang bekerja dengan cara mencari nilai terkecil pada deretan data yang tersisa lalu menukarnya ke posisi paling kiri (terdepan) secara bertahap hingga seluruh elemen terurut. Setelah pengurutan selesai untuk satu wilayah, program langsung mencetak hasilnya dalam satu baris, lalu melanjutkan proses yang sama persis untuk daerah-daerah berikutnya.

2. Kerabat Dekat

Source Code:

```
package main

import "fmt"

type arrInt [1000000]int

func selectionSortAsc(T *arrInt, n int) {
    var i, j, idx_min, t int
    i = 1
    for i <= n-1 {
        idx_min = i - 1
        j = i
        for j < n {
            if T[idx_min] > T[j] {
                idx_min = j
            }
        }
    }
}
```

```

        j = j + 1
    }
    t = T[idx_min]
    T[idx_min] = T[i-1]
    T[i-1] = t
    i = i + 1
}
}

func selectionSortDesc(T *arrInt, n int) {
    var i, j, idx_max, t int
    i = 1
    for i <= n-1 {
        idx_max = i - 1
        j = i
        for j < n {
            if T[idx_max] < T[j] {
                idx_max = j
            }
            j = j + 1
        }
        t = T[idx_max]
        T[idx_max] = T[i-1]
        T[i-1] = t
        i = i + 1
    }
}

func main() {
    var n, m, input int
    var ganjil, genap arrInt

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var nGanjil, nGenap int
        fmt.Scan(&m)

        for j := 0; j < m; j++ {
            fmt.Scan(&input)
            if input%2 != 0 {
                ganjil[nGanjil] = input
                nGanjil++
            } else {
                genap[nGenap] = input
                nGenap++
            }
        }

        selectionSortAsc(&ganjil, nGanjil)
        selectionSortDesc(&genap, nGenap)

        firstPrint := true
        for j := 0; j < nGanjil; j++ {
            if firstPrint {
                fmt.Printf("%d", ganjil[j])
                firstPrint = false
            } else {
                fmt.Printf(" %d", ganjil[j])
            }
        }
    }
}

```

```

        for j := 0; j < nGenap; j++ {
            if firstPrint {
                fmt.Printf("%d", genap[j])
                firstPrint = false
            } else {
                fmt.Printf(" %d", genap[j])
            }
        }
        fmt.Println()
    }
}

```

Output:

```

49  fmt.Scan(&n)
50
51  for i := 0; i < n; i++ {
52      var nGanjil, nGenap int
53      fmt.Scan(&m)
54
55      for j := 0; j < m; j++ {
56          fmt.Scan(&input)
57          if input%2 != 0 {
58              ganjil[nGanjil] = input
59              nGanjil++
60          } else {
61              genap[nGenap] = input
62              nGenap++
63          }
64      }
65
66      selectionSortAsc(&ganjil, nGanjil)
67      selectionSortDesc(&genap, nGenap)
68
69      firstPrint := true
70      for j := 0; j < nGanjil; j++ {
71          if firstPrint {
72              fmt.Printf("%d", ganjil[j])
73              firstPrint = false
74          } else {
75              fmt.Printf(" %d", ganjil[j])
76          }
77      }
78
79      for j := 0; j < nGenap; j++ {

```

Output window shows:

```

Rafi Ramadhan
109082500140

```

Terminal output:

```

PS C:\alpro2> go run soal1.go
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 8 4 2
8 4 2

```

Deskripsi Program:

Program kerabat dekat ini bertujuan untuk mengurutkan nomor rumah kerabat Hercules di berbagai daerah dengan aturan khusus, yaitu menampilkan nomor ganjil terlebih dahulu secara membesar (ascending), kemudian diikuti nomor genap secara mengecil (descending). Sesuai dengan petunjuk soal, saat data dibaca, program akan memisahkan nomor rumah ganjil dan genap ke dalam dua array yang berbeda. Setelah itu, program menggunakan algoritma Selection Sort untuk mengurutkan masing-masing array tersebut sesuai dengan kondisi yang diminta. Terakhir, program mencetak isi array ganjil yang sudah terurut, dilanjutkan langsung dengan isi array genap dalam satu baris yang sama untuk setiap daerahnya.

3. Median

Source Code:

```
package main

import "fmt"

type arrInt [1000000]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]
        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrInt
    var n, input int

    n = 0

    for {
        fmt.Scan(&input)

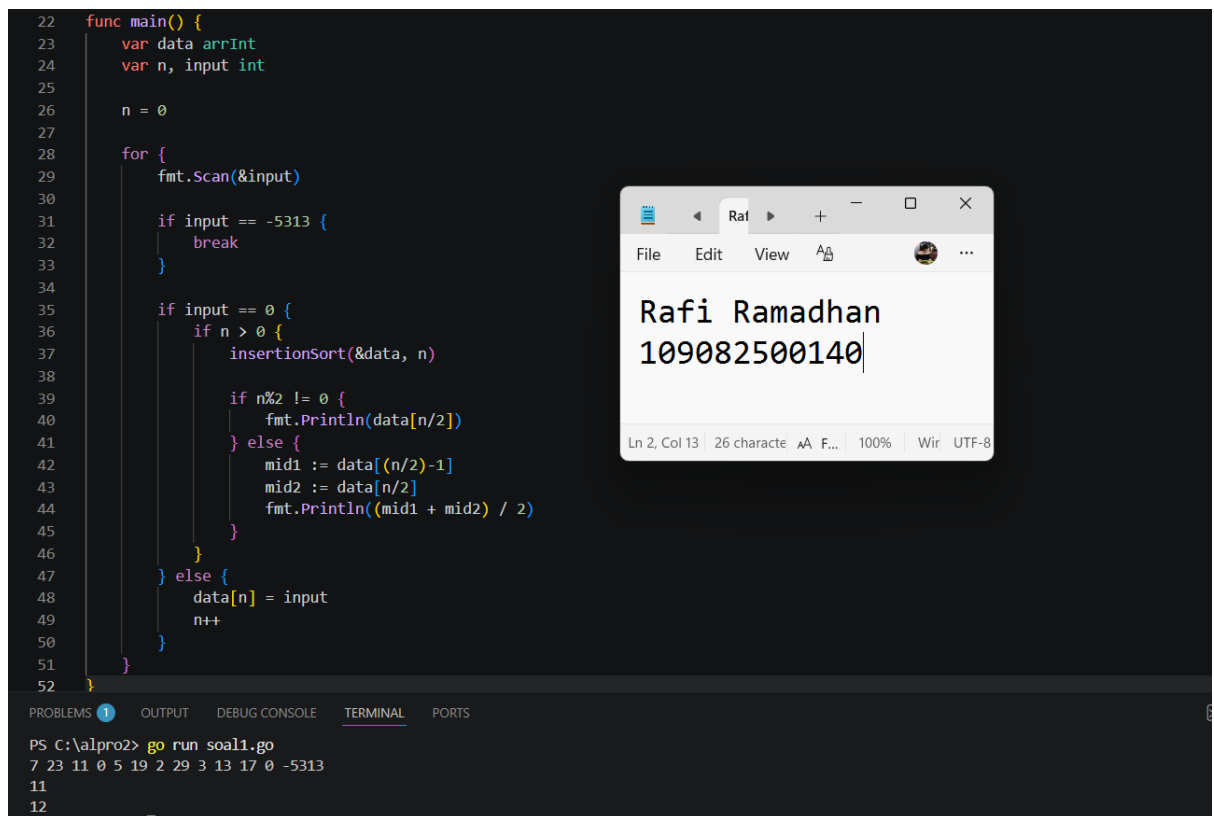
        if input == -5313 {
            break
        }

        if input == 0 {
            if n > 0 {
                insertionSort(&data, n)

                if n%2 != 0 {
                    fmt.Println(data[n/2])
                } else {
                    mid1 := data[(n/2)-1]
                    mid2 := data[n/2]
                    fmt.Println((mid1 + mid2) / 2)
                }
            }
        } else {
            data[n] = input
            n++
        }
    }
}
```

Output:

```
22 func main() {
23     var data arrInt
24     var n, input int
25
26     n = 0
27
28     for {
29         fmt.Scan(&input)
30
31         if input == -5313 {
32             break
33         }
34
35         if input == 0 {
36             if n > 0 {
37                 insertionSort(&data, n)
38
39                 if n%2 != 0 {
40                     fmt.Println(data[n/2])
41                 } else {
42                     mid1 := data[(n/2)-1]
43                     mid2 := data[n/2]
44                     fmt.Println((mid1 + mid2) / 2)
45                 }
46             }
47         } else {
48             data[n] = input
49             n++
50         }
51     }
52 }
```



```
PS C:\alpro2> go run soal1.go
7 23 11 0 5 19 2 29 3 13 17 0 -5313
11
12
```

Deskripsi Program:

Program median ini dirancang untuk membaca sekumpulan bilangan bulat positif satu per satu ke dalam sebuah *array* dan akan berhenti membaca ketika menerima angka -5313. Sesuai instruksi soal, setiap kali program mendeteksi angka 0 pada masukan, program akan langsung mengurutkan seluruh data yang telah terkumpul saat itu secara membesar (*ascending*) menggunakan algoritma *Insertion Sort*. Setelah data terurut, program akan menghitung dan mencetak nilai median (nilai tengah) dari koleksi data tersebut; jika jumlah data ganjil maka nilai yang berada tepat di tengah akan dicetak, sedangkan jika jumlah datanya genap maka program akan menjumlahkan dua nilai tengah lalu membaginya dua, di mana hasil pembagiannya otomatis dibulatkan ke bawah oleh operasi pembagian integer pada bahasa Go.

DAFTAR PUSTAKA

Tim Penyusun Modul. (2023). *Modul Praktikum Algoritma dan Pemrograman 2*. Fakultas Informatika, Telkom University.